

Threads

Part I

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Threads vs. Processes

■ Processes

- Independent execution units with separate memory spaces
- Communication via inter-process communication (IPC) mechanisms
- Higher overhead when creating and switching

■ Threads

- Lightweight execution units **within the same process**
- Share the same address space, enabling faster communication
- Lower overhead for creation and context switching

A process with multiple threads

- **Threads share most of the process resources**
 - The same code, heap, global data, and kernel context
- **Private to each thread**
 - Stack, CPU registers, Program Counter, Thread ID

Thread 1 (main thread)

stack 1

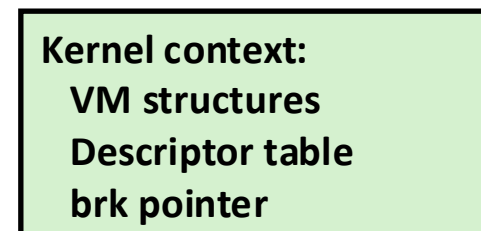
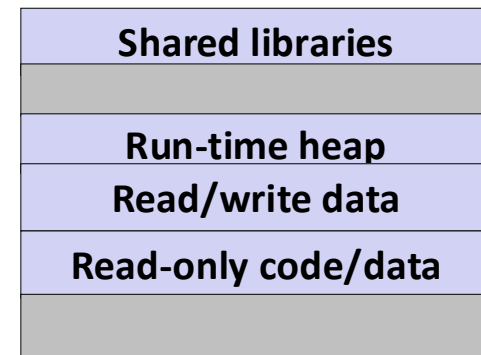
Thread 1 context:
Data registers
Condition codes
 SP_1
 PC_1

Thread 2 (peer thread)

stack 2

Thread 2 context:
Data registers
Condition codes
 SP_2
 PC_2

Shared code and data



Advantages

- Facilitate structuring programs into multiple execution units
- Remove the overhead of multiple address spaces
- Enable more efficient synchronization mechanisms
- Context switching between threads is usually cheaper because they share the same address space
- Threads exist within the same process and there is no parent-child hierarchy like processes

Challenges

■ Shared address space

- Risk of unintended modifications to shared variables

■ Synchronization responsibility

- The programmer must ensure proper synchronization of shared data

■ Thread safety concerns

- Not all library functions are thread-safe
- e.g., `printf` **where simultaneous calls from multiple threads can result in interleaved or corrupted output**

■ Process-wide impact

- A crash in one thread can bring down the entire process

Basic POSIX thread management in Linux

■ `pthread_create()`

- Create a new thread with a function to execute

■ `pthread_self()`

- Get the ID of the calling thread

■ `pthread_join()`

- Wait for a specific thread to exit and collect its status

■ `pthread_exit()`

- End the calling thread

POSIX threads in Linux

- **POSIX thread functions require compiling and linking with the `-pthread` option**
 - This ensures the correct thread-support options are enabled

```
$ gcc -Wall -Wextra -g -o my_program my_program.c -pthread
```

The `pthread_create` function

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

■ A new thread is spawned within the process

- Execution begins at `start_routine`, receiving `arg` as its sole parameter
- The thread does not return from `start_routine`

■ Returns

- 0, if successful
- An error number, in case of error

The `pthread_create` function

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

- **The `thread` parameter is the identifier of the newly created thread**
 - Unlike process IDs (PIDs), thread IDs are unique only within their parent process
- **`attr` are the thread attributes (e.g., priority, scheduling policy, stack size, etc.)**
 - To use default attributes, pass `NULL`

The `pthread_create` function

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

- **`start_routine` is the function executed by the thread**
 - Runs concurrently with other threads in the process
- **`arg` is a single argument passed to the thread's function**
 - If multiple arguments are needed, they should be encapsulated in a structure

The `pthread_self` function

```
#include <pthread.h>

pthread_t pthread_self(void);
```

- **Returns the thread ID of the calling thread**

- This ID matches the value assigned when the thread was created using `pthread_create()`

The `pthread_equal` function

```
#include <pthread.h>

int pthread_equal(pthread_t t1, pthread_t t2);
```

■ Compares thread IDs (`t1` and `t2`)

- Necessary for portability, as thread IDs are opaque and should not be compared directly

■ Returns

- Nonzero, if `t1` and `t2` are equal
- 0, otherwise

The `pthread_exit` function

```
#include <pthread.h>

void pthread_exit(void *retval);
```

■ Terminates the calling thread

- The address in `retval` can be retrieved by **any other thread** within the same process using `pthread_join()`

■ Accessing the local variables of a thread after it has terminated results in **undefined behavior**

The `pthread_join` function

```
#include <pthread.h>

int pthread_join(pthread_t id, void **retval);
```

- **Waits for a specific thread to terminate**
 - **Blocks** the calling thread until the thread identified by `id` terminates
- `retval` is a **pointer** to the value provided by the terminated thread via `pthread_exit()`

Example – Basic thread management

```
void* thread_func(void *arg){
    char buffer[100];
    snprintf(buffer, sizeof(buffer), "I am thread %lu and I am executing\n",
(unsigned long)pthread_self());
    write(STDOUT_FILENO, buffer, strlen(buffer));
    pthread_exit((void*)NULL);
}

int main(){
    pthread_t threads[5];
    int i;

    for(i=0;i<5;i++)
        pthread_create(&threads[i], NULL, thread_func, NULL);

    write(STDOUT_FILENO, "All threads created!", 20);

    for(i=0;i<5;i++)
        pthread_join(threads[i], NULL);

    printf("All threads terminated\n");
}
```

Important notes

- **The order in which threads execute, and thus the output of the `write()` statements, is non-deterministic**
 - As threads, like processes, run concurrently, the order of prints (e.g., “I am thread...” and “All threads created”) will vary across executions
- **The only guarantee is that the “All threads terminated” sentence will always be the last one printed**
 - As it follows the `pthread_join()` calls, ensuring all threads have terminated before execution continues
 - Using `printf()` after `pthread_join()` is safe because all worker threads have terminated

Thread safety considerations

■ Thread routines must call thread-safe functions

- A function is thread-safe if concurrent calls from multiple threads do not produce incorrect results
- Problems often arise when functions use static or global variables assuming single-threaded execution
- If shared variables are required, access must be protected using synchronization mechanisms (e.g., mutexes)

■ Output functions

- With `printf()`, its output is not guaranteed to be atomic
- Calls from different threads may interleave, producing mixed output
- For writes smaller than `PIPE_BUF`, POSIX guarantees that `write()` is atomic, preventing interleaving

Basic synchronization

- **pthread_join()** provides a simple synchronization mechanism between threads
 - Blocks the calling thread until the specified thread terminates
 - Allows a thread to wait for the completion of another thread
- **Any thread may wait for any other thread**
 - Unlike process wait functions, threads are not restricted by parent–child relationships
- **Because threads share the same address space, they can coordinate execution more flexibly**
 - More advanced synchronization mechanisms, such as mutexes and condition variables, will be introduced in the next class

Termination issues

- A process terminates only when **its last thread terminates**
- A thread terminates when:
 - The `start_routine` function returns
 - It calls `pthread_exit()`
 - It is canceled by another thread using `pthread_cancel()`
- If a process terminates, all its threads terminate
 - E.g., the process calls `exec()` or `exit()` or receives a signal causing termination
 - If `main()` terminates before the other threads, and does not call `pthread_exit()`, the process and all its threads are terminated

Example – Termination issues

```
void* thread_func(void *arg){
    char buffer[100];
    snprintf(buffer, sizeof(buffer), "I am thread %lu and I am executing\n",
(unsigned long)pthread_self());
    write(STDOUT_FILENO, buffer, strlen(buffer));
    pthread_exit((void*)NULL);
}

int main(){
    pthread_t threads[5];
    int i;

    for(i=0;i<5;i++){
        /* these threads may not start execution at all */
        pthread_create(&threads[i], NULL, thread_func, NULL);
    }

    /* the process exits automatically, as the end of main triggers an implicit exit(0) */
}
```

Example – Termination issues

```
void* thread_func(void *arg){
    char buffer[100];
    snprintf(buffer, sizeof(buffer), "I am thread %lu and I am executing\n",
(unsigned long)pthread_self());
    write(STDOUT_FILENO, buffer, strlen(buffer));
    pthread_exit((void*)NULL);
}

int main(){
    pthread_t threads[5];
    int i;

    for(i=0;i<5;i++){
        pthread_create(&threads[i], NULL, thread_func, NULL);
    }

    /* the main thread terminates, but the process remains active until all remaining threads have finished
their execution */
    pthread_exit(NULL);
}
```

Passing arguments

- **pthread_create()** allows passing only one argument to the thread function
 - The argument must be a pointer of type `void *`
 - Typically, the address of a variable is passed and cast to `void *`

```
int main(){
    pthread_t threads[2];

    int i = 42;
    pthread_create(&threads[0], NULL, thread_func1, (void*)&i);

    char str[] = "SCOMP";
    pthread_create(&threads[1], NULL, thread_func2, (void*)str);

    ...
}
```

Receiving arguments

- **Inside the thread function, the pointer must be cast back to the appropriate type**
 - For multiple arguments, encapsulate them in a struct and cast the pointer accordingly

```
void* thread_func1(void *arg){  
    int value = *((int*)arg);  
    ...  
}  
  
void* thread_func2(void *arg){  
    char *str = (char*)arg;  
    ...  
}
```

Issues when passing arguments

- Threads start execution and are scheduled in a non-deterministic manner
- Arguments passed to a thread are **pointers**, not copies of the values
- Ensure the pointed value is not modified by other threads before it is used
 - (including the main thread)

Issues when passing arguments

```
void* thread_func(void *arg){
    /* thread receives the argument's address, not its value */
    int value = *((int*)arg);
    ...
}

int main(){
    pthread_t threads[5];
    int i;

    for(i=0;i<5;i++){
        /* value of i can be changed by main before being read by the new thread */
        pthread_create(&threads[i], NULL, thread_func, (void*)&i);
    }

    ...
}
```

A simple solution

- **Store each argument in a separate array element**
 - Ensuring each thread receives a distinct and stable memory location

```
void* thread_func(void *arg){
    int value = *((int*)arg);
    ...
}

int main(){
    pthread_t threads[5];
    int i, args[5];

    for(i=0;i<5;i++){
        args[i] = i;

        /* Each thread receives a different address */
        pthread_create(&threads[i],NULL,thread_func, (void*)&args[i]);
    }

    ...
}
```

A better solution

■ Allocate the argument on the heap

- Ensuring each thread receives a stable memory location that remains valid until explicitly freed

```
void* thread_func(void *arg){
    int value = *((int*)arg);
    free(arg) ;
    ...
}

int main(){
    pthread_t threads[5];
    int i, *arg;

    for(i=0;i<5;i++){
        arg = malloc(sizeof(int)) ;
        *arg = i;

        /* each thread gets its own stable memory location that lives until it explicitly frees it */
        pthread_create(&threads[i],NULL,thread_func, (void*) arg) ;
    }
    ...
}
```

Exit value

- The exit value passed to `pthread_exit()` must be provided by its address
- Do not use addresses of local variables, as they no longer exist after the thread terminates
 - Local variables are stored in the stack, which is reclaimed once the thread ends
 - Accessing such memory after termination results in **undefined behavior**, as the memory space might be reused

Issues when exiting

- The stack is reclaimed when the thread terminates, leaving an invalid pointer

```
void* thread_func(void *arg) {
    int x = 45;
    ...
    /* Incorrect: returns address of stack variable */
    pthread_exit((void*) &x);
}

int main(){
    pthread_t thread;
    void *ret;
    int value;
    ...
    pthread_join(thread, &ret);

    /* Address is no longer valid */
    value = *((int*) ret);
    printf("Exit value: %d\n", value);
    return 0;
}
```

Issues when exiting

- **The exit value must be a valid, persistent memory address**
- **Allowed options**
 - Heap allocation – use `malloc()` to allocate memory for the value
 - Global and `static` variables – simpler, but not thread-safe for multiple threads
- **Never use**
 - Local variables (stack is reclaimed after thread exits)
 - Direct integer casts (`(void*) 45`) – non-portable and unsafe

Using the heap

```
typedef struct{
    int a;
    float b;
}my_struct;

void* thread_func(void *arg){
    my_struct *ms = (my_struct*)malloc(sizeof(my_struct));
    ms->a = 10;
    ms->b = 12.0;
    pthread_exit((void*)ms);
}

int main(){
    void *ret;
    my_struct values;

    ...

    pthread_join(thread, (void*)&ret);
    values = *((my_struct*)ret);
    free(ret);
    printf("%d %f\n", values.a, values.b);
    return 0;
}
```

Exercise 1

- **Implement a multi-threaded program to find the maximum value in an array $v[1000]$ filled with random numbers**
 - Use another array $max[10]$ to store partial maximums, one per thread
- **Create 10 threads. Each thread:**
 - Receive its thread index as an argument
 - Processes 1/10 of $v[]$
 - Identifies the maximum in its assigned segment
 - Stores its result in $max[i]$ (where i is the thread index)
- **The main thread:**
 - Waits for all threads to finish
 - Determines the global maximum from $max[]$